

03 — Array of Structures: Uma Introdução

Introduzido, não desenvolvido. A resposta precisa de medições reais de GPU, que é a Phase 2. O objetivo aqui é tornar a pergunta visível.

Duas formas de guardar o mesmo dado

Suponha 1000 partículas, cada uma com posição e velocidade.

AoS — Array of Structures. A intuitiva:

```
struct Particle {
    float x, y, z;
    float vx, vy, vz;
};

Particle particles[1000];
```

memória:

```
+---+---+---+---+---+---+---+---+---+---+
| x0 | y0 | z0 | vx0 | vy0 | vz0 | x1 | y1 | z1 | ... |
+---+---+---+---+---+---+---+---+---+---+
\_____ partícula 0 _____/\_____ partícula 1 ...
```

SoA — Structure of Arrays. A que favorece hardware paralelo largo:

```
struct Particles {
    float x[1000];
    float y[1000];
    float z[1000];
    float vx[1000], vy[1000], vz[1000];
};
```

memória:

```
+---+---+---+---+---+---+---+---+---+---+
| x0 | x1 | x2 | ... | y0 | y1 | y2 | ... |
+---+---+---+---+---+---+---+---+---+---+
\_____ todo x _____/\_____ todo y _____/
```

Por que a escolha importa

Considere um loop que lê só **x** de cada partícula.

AoS: os valores de x estão 24 bytes distantes um do outro. Cada cache line de 64 bytes buscada contém no máximo 3 floats úteis de 16 — o resto é dado de y , z e velocidade que não foi pedido.

SoA: os valores de x são adjacentes. Toda linha buscada é inteiramente útil.

O caso inverso também existe: código que toca *todos* os campos de *uma* partícula por vez favorece AoS, porque tudo que precisa chega junto.

A regra geral

Padrão de acesso	Melhor layout
Tocar todos os campos de um elemento	AoS
Tocar um campo de todos os elementos	SoA

Hardware paralelo largo — SIMD numa CPU, warps numa GPU — está esmagadoramente na segunda categoria. É por isso que SoA é a resposta usual em código de GPU, e por isso que tensores de deep learning são, na prática, SoA por construção.

Onde isso vai dar

A versão em GPU tem uma penalidade muito mais severa. Quando 32 threads de um warp leem, cada uma, um campo de um struct, um layout AoS espalha essas 32 leituras pela memória e os acessos não fazem coalescing. O mesmo dado em forma SoA é 32 floats contíguos — uma transação limpa só.

Não desenvolva isso mais ainda. Medir isso direito exige uma GPU e um profiler. O que importa agora é que a pergunta existe e que é uma pergunta de *layout*, respondível olhando pra endereços.

Por que isso importa para CUDA

Diretamente. Essa é uma das decisões de layout padrão em GPU, e ela reaparece em todo nível:

- Simulações de partícula e kernels de física escolhem explicitamente entre os dois.
- Tensores de deep learning são SoA por natureza — um batch de ativações é um array contíguo de um "campo" só.
- **NCHW versus NHWC** é esse mesmo argumento pra tensores de imagem: qual eixo é contíguo decide quais padrões de acesso fazem coalescing.
- O North Star de pesquisa deste repositório — kernels fundidos de normalização — é em parte um argumento de layout. Como um tensor está organizado decide se um kernel fundido consegue lê-lo eficientemente.

[experiments/memory-access/](#) é onde isso é medido em hardware real. Este arquivo é onde a pergunta é feita pela primeira vez.